CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool

Marko Ivanković
Universität Passau
Passau, Germany
marko@ivankovic.me

ABSTRACT

A syntax-aware diff maps one abstract syntax tree (AST) onto another, and every evaluation of one rests on three assumptions: that computing the mapping is affordable, that the correct mapping is unique, and that the mapping is what the reader sees. This paper tests all three, and finds each of them false often enough to change how such tools should be measured.

We first measure what exactness costs. Across 2,922 file pairs sampled from 100 open-source repositories, a single whole-tree tree-edit-distance computation completes within a one-second budget for 54.5% of code pairs, and the collapse falls between two adjacent, entirely ordinary file sizes. We then turn to the ground truth itself, with no diffing tool involved: a corpus of 468 real-world changes in 24 languages, each carrying a human-authored AST mapping in which a set of interchangeable nodes can be recorded as admitting several equally correct pairings, and 43 of them carrying a human-painted rendering of the same change. In 15.3% of the changes annotated with the facility to record it, no single mapping is the correct one; in a rarer class, no one-to-one mapping is correct at all, because the true correspondence is $N{:}M$. 31.8% of AST nodes carry no text of their own and never reach the screen, so a node-level metric scores heavily on decisions no reader can see. And where the rendering was painted by hand, 44.2% of changes admit two equally correct renderings of one settled mapping. Together these bound what any evaluation scoring a tool against one fixed answer can report.

Against that ground truth we measure four established tools—GumTree, Unix `diff`, difftastic, and diffsitter—and find line-level mismatch rates from 1.111% to 4.846%, with no AST-aware tool among them beating plain line-based `diff`. Finally, as a system contribution, we present CodeDiff, a tree-sitter-based diffing tool that combines APTED's exact tree edit distance, scoped to bounded subtree pairs rather than whole files, with GumTree's move-recovery heuristic and Myers' sequence diff into one five-phase pipeline, engineered against the measurements above rather than against a worst-case bound, and mapping 99.94% of AST nodes correctly on the same corpus.

CCS CONCEPTS

• **Software and its engineering** → **Software maintenance tools**; *Empirical software validation*.

KEYWORDS

source code differencing, syntax-aware diffing, empirical software engineering, source code tree edit distance

ACM Reference Format:
Marko Ivanković. 2026. CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference'26).* ACM, New York, NY, USA, 14 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Code diffing, the process of comparing two pieces of source code and computing the set of operations that transforms one into the other, underlies modern version control and, especially, modern code review. During a review, developers read the diff to see what changed and what stayed the same, so the quality of the diff directly shapes the quality of the review.

The standard diff used by almost all developers and platforms today is line-based: the file is treated as a sequence of text lines, and the tool computes a shortest edit script between the two sequences using three operations, insert, delete, and update. This approach is computationally cheap and fast even on modest hardware. However, most implementations offer no move operation at all, and the tools that do support one support it only in limited ways. A whole class of common changes—extracting code into a reusable function, wrapping a block in a condition, reordering declarations—is ultimately a relocation of existing code, and without a move operation every such change must be rendered as a full deletion plus a full insertion. Reconstructing the correspondence between the deleted and inserted halves is then left to the reviewer, by eye, which is exactly the kind of mechanical, attention-taxing work humans do least reliably.

One way to compute diffs with a genuine move operation is to compare the code's abstract syntax tree (AST) instead of its raw text. An AST diff maps nodes of one tree onto

Conference'26, June 03–05, 2026, Ludbreg, Croatia

ACM ISBN 978-1-4503-XXXX-X/26/06
https://doi.org/XXXXXXX.XXXXXXX

nodes of the other; a move is a mapping whose nodes match but whose ancestry changed. Tree diffing is a well-studied field, and Section 2 surveys the work this paper builds on. In practice, however, tree diffing has seen little adoption, and we argue the central reason is cost: the best exact tree-diffing algorithms are $O(n^3)$ in tree size, and Section 3 measures directly how often a whole-tree computation at real-world file sizes exceeds an interactive time budget.

This paper is about the three assumptions that sit underneath every such tool, and about what happens to its evaluation when they fail. The first is that computing an AST mapping is affordable: the best exact tree-diffing algorithms are $O(n^3)$ in tree size, and Section 3 measures directly how often a whole-tree computation at real-world file sizes exceeds an interactive budget. The second is that the mapping a tool should recover is unique. The third is that the mapping is what the reader sees. Sections 4 and 5 test the second and third against a corpus of human-authored ground truth, without reference to any diffing tool at all, because they are properties of code change rather than of any implementation. Only then, in Section 6, do we measure what existing tools recover, and in Section 7 present a tool of our own.

The paper is organized around six research questions, each answered by the correspondingly numbered research answer, RA1–RA6, set in a box at the point in the paper where the measurement supporting it is reported. The first four name no diffing tool: they are about what the problem costs and what a correct answer to it even is.

- **RQ1 (Cost of exactness).** What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation—the algorithmic core every exact, syntax-aware diff tool is built on—complete within a one-second budget? Answered by RA1, Section 3.
- **RQ2 (Uniqueness).** When does a real-world code change have no single correct mapping between the two syntax trees—because several one-to-one mappings are equally correct, or because the true correspondence is not one-to-one at all? Answered by RA2, Section 5.
- **RQ3 (Visibility).** What share of an AST mapping can a rendered diff display at all, and how much of it is therefore scaffolding a reader never sees? Answered by RA3, Section 5.
- **RQ4 (Rendering).** Once the mapping is settled, is the rendering that follows from it unique—or does one correct mapping admit several equally correct renderings? Answered by RA4, Section 5.
- **RQ5 (Tool accuracy).** How accurately do current state-of-the-art code-diffing tools recover real-world code changes, measured against human-authored ground-truth mappings of those changes? Answered by RA5, Section 6.
- **RQ6 (Tool cost).** What does each of those tools cost to run on the same corpus, and how does that cost distribute across easy and hard changes? Answered by RA6, Section 6.

This paper makes four contributions, one empirical, one dataset, one evaluation, and one system:

- **Empirical study.** An empirical study of 100 real-world open-source repositories, about 1.35 million files in 30 languages, showing that the worst-case inputs tree-diffing algorithms are analyzed against are rare in practice, but that the tail is real and large enough that a production tool must handle inputs beyond what a whole-tree tree-edit-distance computation can process within an interactive budget.
- **Dataset.** A corpus of 468 real-world code changes in 24 languages, each carrying a human-authored ground-truth AST mapping in which every node is labeled with one of seven operations—and in which a set of interchangeable nodes can be recorded as admitting several equally correct pairings rather than forced to one, which is what makes RQ2 measurable. 43 of those changes additionally carry a human-painted *rendering*: the spans of text a reader should see marked, independently of which AST nodes carry them, recorded once or twice according to whether the annotator judged the rendering unique. Both were produced with a purpose-built annotation tool (Section 4), and the methodology is reproducible from the project's research repository.
- **Evaluation.** A measurement of what that ground truth can and cannot ask of a tool: how often it admits no unique mapping, how much of it a reader ever sees, and how often its rendering is itself ambiguous—together bounding what any evaluation scoring a tool against one fixed answer can report—followed by a quantified comparison of four established diffing tools, GumTree, Unix `diff`, difftastic, and diffsitter, against it on accuracy and on cost.
- **System.** CodeDiff, a five-phase diffing pipeline built from APTED tree edit distance, scoped to bounded subtree pairs rather than to whole files, together with GumTree's move-recovery heuristic and Myers' sequence diff, engineered as one production tool against the measurements above rather than against a worst-case bound.

2 BACKGROUND

This section introduces the algorithms and tools the rest of the paper depends on: the algorithms CodeDiff builds its pipeline from, and the tools it is evaluated alongside. Section 8 returns to the closest prior work in more depth once the paper's own results are in hand.

2.1 Algorithms

Zhang-Shasha [9] gives the classic dynamic-programming algorithm for ordered tree edit distance. Given two rooted, ordered trees, it computes the minimum-cost sequence of node insertions, deletions, and relabelings that transforms one tree into the other, together with the mapping that realizes that cost. It is exact, but its keyroot decomposition does repeated work across overlapping subproblems. Later

algorithms reduce this overhead while keeping the same exact result.

APTED [8] is the state-of-the-art exact tree-edit-distance algorithm. It improves on Zhang-Shasha by choosing, for each subproblem, the cheapest of three decomposition strategies (left path, right path, or heaviest inner path), rather than a single fixed strategy. This choice provably minimizes the total work across the whole computation. APTED still costs $O(n^3)$ in the worst case for general trees, the same asymptotic bound as Zhang-Shasha, but its real-world runtime is markedly lower, because most real trees are far from the adversarial shape that bound assumes.

GumTree [4] targets source code directly, not general trees. Instead of one global tree-edit-distance computation, it runs a top-down phase that greedily matches large isomorphic subtrees, then a bottom-up phase that matches remaining container nodes by the Dice coefficient of their already-matched descendants. GumTree also introduces a distinct edit operation, move, and a recovery pass that pairs up deleted and inserted identical subtrees the top-down and bottom-up phases missed. This design favors speed over exactness, at the scale real source files require.

Myers' O(ND) algorithm [7] solves a different, more restricted problem: the shortest edit script between two flat sequences, not two trees. It underlies most line-based diff tools, including Unix `diff`. Its cost scales with the size of the edit script, not the size of the inputs, so it stays fast when two sequences are similar, exactly the common case for a version-controlled file.

2.2 Tools

difftastic [5] and **diffsitter** [3] are a newer generation of practical, tree-sitter-based diffing tools, the same parsing layer CodeDiff itself is built on. Both prioritize a diff a human reads comfortably over an exact node-to-node mapping: difftastic reduces the parse tree to an s-expression and aligns it with its own structural algorithm, and diffsitter applies a related tree-sitter-based alignment over a smaller, hand-picked set of compiled-in grammars. Neither computes a full tree-edit-distance mapping the way APTED or GumTree do, so neither directly targets the ground-truth-accuracy metric this paper measures against in Section 6—a difference in design goal, not a shortcoming, and the reason we include both as companions in the same problem space rather than as head-to-head competitors on identical terms.

3 THE COST OF EXACTNESS

Formal treatments of the algorithms in Section 2 report worst-case asymptotic complexity: APTED and Zhang-Shasha are both $O(n^3)$ for general trees, where n is tree size. This analysis is necessary, but it is not sufficient for a production tool. GumTree's own documentation states that it targets research use, not production deployment at scale—a reasonable scope for its own goals, but one that leaves open a question every production diff tool must answer:

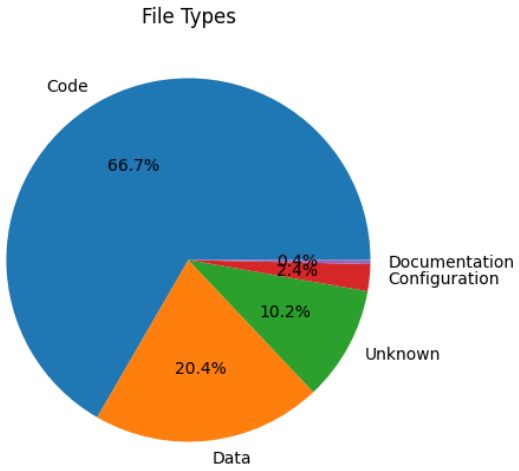

Figure 1: Distribution of file types across the corpus.

RQ1 (Cost of exactness), stated in Section 1 and restated here: what percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation—the algorithmic core every exact, syntax-aware diff tool is built on—complete within a one-second budget?

A tool engineered only against a worst-case bound risks two failure modes. It can over-engineer for inputs that never occur in practice. It can under-engineer for a real tail its asymptotic analysis never quantified. Answering RQ1 needs measurement, not further asymptotic analysis.

We measure this directly. We built a corpus of 100 open-source repositories, drawn from the Gentoo Linux distribution's package list and popular repositories on GitHub. We cloned each repository to a depth of 50 commits from every branch tip and parsed every file with Tree-sitter, recording byte size, line count, language, and AST node count per file. The depth limit matters for what follows: it bounds how far back the commit sampling in this section can reach, and a sampled (repository, commit, path) triple stays resolvable only while that commit remains within the depth window of the local clone. The full methodology, and every per-language number, is reproducible from the project's research repository; here, we summarize the numbers that bound what any syntax-aware diff tool has to process.

The corpus has 1,347,490 files across 30 languages. About two-thirds of all files are code. The rest is configuration, data, and documentation (Figure 1). Restricted to code files, Table 1 reports size percentiles in bytes, lines of code, and AST nodes.

The largest file in the corpus has 573,334 AST nodes. Byte size and AST node count have a strong correlation, Pearson $r = 0.8794$, which simplifies to $|\text{AST}| \approx \text{bytes}/5$ for quick estimates. Arafat and Riehle report that 47.5% of all commits touch 10 lines or fewer [1], while commits of over 10,000 lines still occur.

Table 1: Per-file size percentiles across all languages, code files only.

Metric	p50	p90	p99	p99.9	Max
Bytes	1,508	13,945	90,132	464,325	23,949,786
LOC	42	378	2,244	7,818	65,563
AST nodes	224	2,868	18,553	70,828	573,334

Measurement. To answer RQ1's percentage directly, we sampled 2,922 real (before, after) file pairs from single-parent commits in these repositories, stratified per language and per size bucket so that large files are represented rather than drowned out by the far more common small ones. We ran a single whole-tree tree-edit-distance computation on every pair—one APTED implementation, applied to the full before and after trees with no pipeline heuristics of any kind around it—under a hard one-second budget enforced by terminating the computation's process. This implementation includes an optimization that makes it faster than a textbook APTED, so the failure rates below are a lower bound on how often a whole-tree computation misses the budget, not an upper bound.

Every sampled pair was checked to still resolve against the local clones before measurement, and all of them did. This is worth stating because the check has failed before: an earlier sample, drawn against a shallower corpus, had decayed to roughly 41% unresolvable by the time it was re-measured, and the losses fell on whole repositories rather than spreading evenly across the sample. A measurement over that sample would have reported a healthy-looking rate while quietly answering a question about a different, smaller corpus.

Figure 2 shows the fraction of pairs completed within one second, per size bucket, split by what kind of artifact the file is: code (1,942 pairs across general-purpose programming languages), configuration, data, and markup formats (700 pairs), and scripting languages (280 pairs). The split matters because the three populations behave differently—blended, the headline number would move with how many configuration files a sample happens to contain, independently of anything the algorithm does to source code.

RA1 (Cost of exactness)

For source code, a whole-tree tree-edit-distance computation is dependable only for small files, and the transition is abrupt rather than gradual: it completes within one second for 97.9% of code pairs of 10–30 lines, but for only 25.3% of code pairs of 100–300 lines, and never recovers above one third in any larger bucket. Across all sampled code pairs, 54.5% complete in time.

What makes that a design problem rather than a tail problem is where the collapse falls: between two adjacent buckets, neither of which is an unusual size for a source file, and both well inside the range ordinary source files occupy (Table 1). The corpus-wide figure is a sample-weighted aggregate: it deliberately over-represents large files and therefore understates the rate a uniform sample of real commits would show, but the per-bucket rates carry the answer regardless of weighting.

The three artifact categories do not behave alike. Scripting languages complete within the budget 83.9% of the time, well clear of code at 54.5% and configuration, data, and markup formats at 49.0%; the latter two are close to each other and separated by less than six points, so the meaningful split is scripting against the rest rather than a clean three-way ordering. Figure 2 shows the same pattern per bucket: scripting stays above every other category in all seven, while configuration and data fall furthest in the large buckets, despite containing the most repetitive and most regularly structured files in the corpus. We conjecture this reflects tree edit distance being sensitive to tree shape rather than to node count alone, a large flat sequence of similar siblings being an unfavorable shape for it. This measurement does not isolate shape from size, so we state it as a conjecture rather than a result.

The common case is genuinely cheap—most files and most commits are small—but the tail is real: files up to 573,334 AST nodes occur in this corpus, and Arafat and Riehle's data shows commits of over 10,000 lines occur in practice. A production diff tool must be engineered for both regimes, and cannot rely on a whole-tree computation alone for either. Section 7 turns this answer into CodeDiff's concrete design targets.

4 A GROUND TRUTH FOR CODE CHANGE

The three sections that follow all measure against one artifact: a corpus of human-authored answers to the question "what changed here?", built without reference to any diffing tool. This section describes how it was made and what it records, because everything Section 5 finds is a property of that record rather than of any algorithm.

Mapping the syntax trees. The corpus holds 468 fixtures spanning 24 languages. Each fixture pairs a before and after version of a source file, representing a realistic code change—a bug fix, a refactor, an added feature, or a performance-motivated rewrite. Most fixtures are captured directly from a specific commit in a real open-source repository; the rest are curated examples of a common, realistic change pattern. For every fixture, a human annotator used a purpose-built tool, `human_solver`, to walk the before and after syntax trees side by side and label every node with one of seven operations: identical, update, match-but-not-identical, insert, insert-with-children, delete, and delete-with-children. Nodes are addressed by a path of kind and sibling position, not by parser-internal node identity, since that identity is not stable across separate parses of the same file.

Where several candidate nodes are genuinely interchangeable, a single pairing would be an arbitrary choice rather than a fact about the change, so the tool also lets the annotator record a *multi-map group*: a set of N before-side nodes

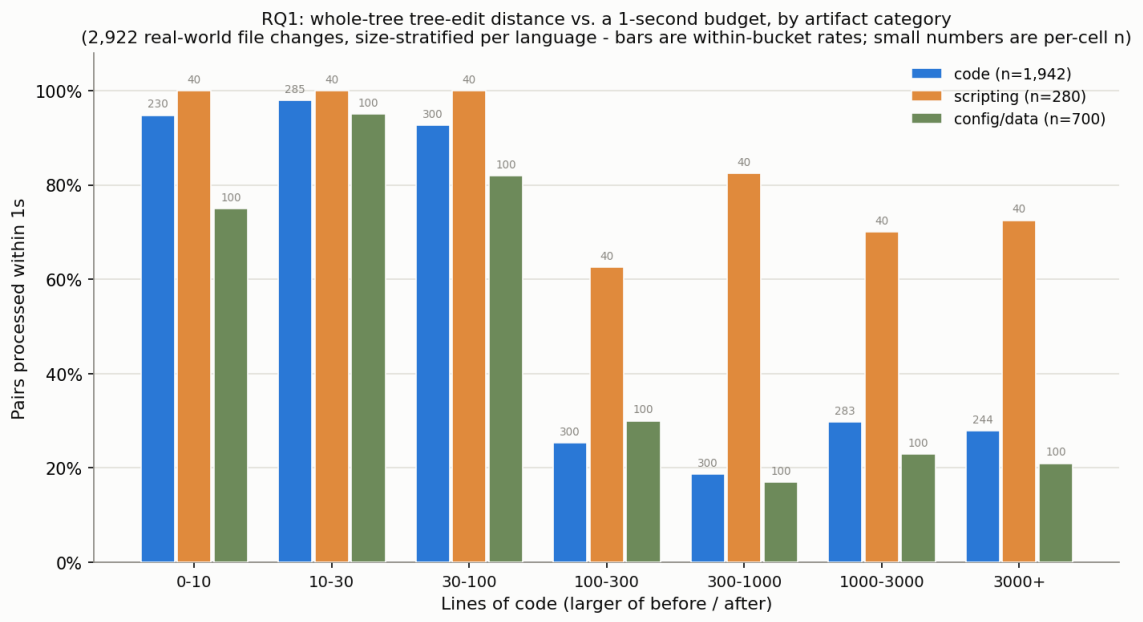

Figure 2: Fraction of sampled file pairs a whole-tree tree-edit-distance computation completes within a one-second budget, per size bucket and artifact category.

and M after-side nodes in which any consistent pairing is correct, scored as correct as long as a tool realizes $\min(N, M)$ pairs and treats the remainder of the larger side as inserted or deleted. A group is not an expression of annotator uncertainty. It is the annotator's finding that more than one mapping is equally right, and it is what RQ2 measures. This human-authored mapping, groups included, is the ground truth every number in this paper is measured against.

Painting the rendering. A mapping between syntax trees is not what a developer reads. A tool must turn its mapping into highlighted regions of text, and that projection is a second decision the mapping does not determine. So a subset of the corpus carries a second, independent ground truth: a *painting*, in which the annotator marks the spans of source text a reader should see highlighted, labelled moved, updated, inserted or deleted, working from the two files rather than from either tree. A painting is authored in the same tool, in a separate view, and it is deliberately not derived from the mapping—it is the answer to "what should this look like", recorded so that the two questions can be scored apart.

Painting is slower per fixture than mapping, and it was done on the small, curated changes first: 43 of the 468 fixtures carry one, all of them from the hand-written portion of the corpus, spanning 7 languages, with a median of 14 lines against 219 for the rest of the corpus. Section 5 states what that population does and does not license.

The annotator records either one painting or two. Two is the interesting case and the reason the facility is shaped this way: the same change often has a tight rendering and a generous one, and both are defensible. A *Minimal* painting leaves indentation, brackets and other pure punctuation unpainted, marking only the content that carries meaning; a *Full* painting accounts for every byte whose role changed. Recording one painting instead is a positive assertion, not an omission—it is the annotator's finding that this change has a single defensible rendering, and the fixture then holds both render modes to that one answer. The count is therefore the datum, exactly as a multi-map group's existence is the datum on the mapping side.

A painting also has one degree of freedom the mapping format lacks. A painted entry carries a *list* of spans per side rather than a single span, so an annotator who finds that three comment lines became one, or that one import became two, can record that correspondence directly instead of approximating it. Section 5 uses this: it is the only place in the corpus where an $N{:}M$ correspondence is written down as what it is.

5 WHAT THE GROUND TRUTH CAN ASK

This section measures the ground truth of Section 4 against itself. No diffing tool appears in it. Every question here is about what a correct answer to a code change even is—whether it is unique, how much of it a reader can see, and whether the rendering that follows from it is unique either—and each answer bounds what any evaluation built on such a corpus, including this paper's own in Section 6, is able to report.

RQ2 (Uniqueness). When does a real-world code change have no single correct mapping between the two syntax trees—because several one-to-one mappings are equally correct, or because the true correspondence is not one-to-one at all?

RQ3 (Visibility). What share of an AST mapping can a rendered diff display at all, and how much of it is therefore scaffolding a reader never sees?

RQ4 (Rendering). Once the mapping is settled, is the rendering that follows from it unique—or does one correct mapping admit several equally correct renderings?

Uniqueness. Any accuracy metric of the usual kind rests on an assumption worth testing before any tool is measured by it: that for a given change there is one correct mapping to recover. RQ2 asks when that assumption holds, and the corpus shows it failing in two different ways, which we keep apart because they have different consequences.

Multiple optimal solutions exist. Several candidate nodes are genuinely interchangeable—one of two identical calls is removed, a member is appended to an expression chain, an argument is added to a list of similar arguments—so more than one one-to-one pairing reproduces the change equally well, and choosing between them is arbitrary rather than correct. A diffing algorithm can still emit a correct answer here; what fails is the assumption that the correct answer is unique.

No one-to-one optimal solution exists. The true correspondence is $N{:}M$: several nodes on one side correspond to one or several on the other, all of them at once. Two call sites are refactored into one, a block of comment lines is condensed into a single line, one import is split across two. Here no one-to-one mapping is correct, however the algorithm chooses, and the qualifier matters: not that no optimal solution exists, but that none in the one-to-one form these algorithms produce reaches the true correspondence.

The first is measurable on this corpus, because the annotation tool records it explicitly as a multi-map group (Section 4). The second is not, and we treat it accordingly below.

One property of the corpus has to be stated before the rate, because it bounds what the rate can mean. The multi-mapping facility was added to the annotation tool partway through corpus construction. 217 fixtures were last annotated before it existed and were never revisited; none of them records an ambiguity, and none of them could, whatever the change actually contained. Table 2 therefore splits the corpus by annotation era rather than reporting one blended rate, which would measure annotation practice as much as the phenomenon.

The middle row is the estimate. Of the 236 fixtures annotated from the start by someone who had multi-mapping available, 36—15.3%—contain at least one place where no single pairing is the correct one. The third row is reported for completeness and excluded from that estimate: those 15 fixtures predate the facility and were reopened afterwards, and a fixture is reopened precisely because something in it needed attention, so their 73.3% rate is selection-biased upward by construction. The corpus-wide 10.0% is a floor rather than a census, for the reason the first row makes explicit.

Table 2: How often the ground truth admits no unique mapping, split by whether the annotator had the multi-mapping facility available. Only the second row is an unbiased estimate; the third is selection-biased and the first is structurally zero.

Annotation era	Fixtures	Ambiguous	Rate
Before the facility existed	217	0	0.0%
Annotated with it available	236	36	15.3%
Predates it, later revisited	15	11	73.3%
Whole corpus (a floor)	468	47	10.0%

The ambiguity is local. Across the 209 groups in the corpus, the median group spans 2 candidate nodes on its larger side and the largest spans 10; exactly 1 extends the ambiguity to whole subtrees rather than to individual nodes. 98.6% of groups have different candidate counts on the two sides, and 55.5% of all groups are the smallest such case, two candidates on one side and one on the other or the reverse. An unequal group usually asks which of N interchangeable nodes is the one that was added or the one that survived—but not always, and the next subsection is about the cases where it asks something else. Ambiguity is also unevenly spread: one fixture alone carries 36 of those groups. The two operations a group can carry split almost evenly, 106 identical against 103 matched-but-not-identical, so ambiguity is not a property of unchanged code alone.

How many fixtures are affected and how much of the ground truth is affected are different quantities, and both are worth stating. Weighted by mapping decisions rather than by fixtures, the groups account for 268 of the 6,786 node pairs the ground truth records outside identical code—3.9%. The two readings are both correct and they answer different questions: a metric aggregated over nodes or lines absorbs an ambiguity of this size almost invisibly, while a metric that asks whether a fixture was mapped exactly right inherits the full 15.3%, because one ambiguous pair is enough to fail a fixture.

No one-to-one optimal solution exists. The second failure is not a matter of degree. An ordered tree edit distance maps each node of one tree to at most one node of the other; that is part of the definition of the edit script it computes, not a limitation of any particular implementation. An $N{:}M$ correspondence is therefore not in the codomain of any algorithm in this family, which is every tool measured in this paper. It cannot be found by a better heuristic, because there is nothing in the output language to find it with. The best such an algorithm can do is pick one pair and render the rest as insertions or deletions—which is not an approximation that happens to score badly, but a statement that is false about the change.

We report this as an existence result, not a rate. Nothing in the *mapping* format marks it: it records one node against one node, so an annotator who meets an $N{:}M$ correspondence records the closest one-to-one approximation and writes the

Table 3: Changes in the corpus whose true correspondence is N:M, found by inspection of annotator commentary rather than by enumeration. "Encoded" is whether the ground truth carries a multi-map group at the site ("Enc."); where it does, that group is the approximation, not the correspondence.

Change	Correspondence	Enc.
License comment condensed	5 comments to 1	yes
Import split in two	1 identifier to 2	yes
Two string literals merged	2 literals to 1	yes
If rewritten as a ternary	2 statements to 1	yes
Two call sites merged	2 statements to 1	no
Two asserts split into six	escape sequences	no

real one in prose, in the fixture's test case. The instances in Table 3 were found by reading those comments, not by enumeration, so they are a lower bound of unknown tightness and no denominator is quoted for them. The painting format is the exception, and RA4 quotes a rate from it: a painted entry carries a list of spans per side, so there the correspondence can be written down as what it is.

Four of the six carry a multi-map group sitting exactly at the N:M site, and reading those groups is what shows the encoding is doing double duty. In the condensed license block the group lists five before-side comment nodes against one after-side comment; in the split import, one before-side identifier against two after-side identifiers. None of the four is a set of interchangeable candidates of which one survived. Every member corresponds, and the group's own semantics—realize $\min(N, M)$ pairs, delete or insert the remainder—assert the opposite. The same encoding therefore carries two different phenomena, and only the annotator's prose separates them. The remaining two instances carry no group at all: the correspondence was noted and left unencoded, which is the more honest of the two options available.

This also qualifies a claim made above. Accepting any consistent pairing within a group removes the scoring error for the first phenomenon exactly, because every pairing it admits really is correct. For the second it does not: it scores the leftover deletions as correct when the truth is that those nodes correspond too. The corpus is right about interchangeability and charitable about N:M. Two of the instances in Table 3 postdate the corpus snapshot every other number in this section is measured against, which is why they appear here and in no rate.

RA2 (Uniqueness)

A correct AST mapping is not always unique. In 15.3% of the fixtures annotated with multi-mapping available (36 of 236), at least one set of nodes admits several equally correct pairings, with the corpus-wide 10.0% standing as a floor; weighted by mapping decisions rather than fixtures, that is 268 of 6,786 non-identical node pairs, 3.9%. An evaluation that scores a diffing tool against one fixed pairing therefore risks marking correct output wrong on roughly one change in six—whenever the tool picks a different, equally valid pairing—and any accuracy metric defined over a single ground-truth mapping carries that error term whether or not it reports it. A second, rarer failure is categorical rather than statistical: where the true correspondence is N:M, no one-to-one mapping is correct at all, and no algorithm whose output is a one-to-one node mapping can express one. We report instances of that rather than a rate, because neither the corpus format nor the algorithms have a way to represent the thing being counted.

Both rates in Table 2 remain lower bounds for a second reason the corpus cannot correct for: a group exists only where an annotator noticed the ambiguity and recorded it. Nothing detects an ambiguity the annotator resolved without noticing it was a choice. That the count is conservative in this direction is what makes it usable—an over-count would be an argument about annotation discipline, whereas an under-count still establishes that the phenomenon is common enough to affect how accuracy is measured.

The practical consequence is a claim about methodology rather than about any tool. Scoring against a single pairing is the default in this area, and on a corpus like this one it puts a measurable error term into every reported rate: on the affected sixth of changes, a tool that picks a different valid pairing is recorded as wrong for a choice that was never wrong. The corpus in this paper avoids that for the first phenomenon by validating any consistent pairing within a group, as Section 4 describes, which is why the accuracy numbers reported in Section 6 do not inherit its error term. For the second it has no such escape, and neither does any corpus built on one-to-one mappings. Reporting a fixed-pairing metric is still defensible, but the ambiguity rate belongs alongside it.

Visibility. RQ2 asks whether the mapping is unique. RQ3 asks a prior question about the same object: how much of it anyone ever sees. A syntax tree contains two kinds of node. Some carry text of their own—an identifier, an operator, a literal, a comment. Others are pure structure: a `block` whose extent is exactly its children's, an `expression_statement` wrapping one expression, the chain of single-child nodes a grammar interposes between a statement and the name inside it. A rendered diff highlights regions of source text, so a node of the second kind cannot appear in the output on its own. Whatever a tool decides about it is invisible unless the decision propagates to a node of the first kind.

This matters because it separates two things a node-level accuracy metric silently merges. A tool that maps every displayed token correctly and every structural wrapper wrongly produces a diff a reader cannot fault; the same tool scores badly on a metric that counts all nodes equally. The converse is worse: a tool can be flattered by a grammar that nests deeply, since each extra layer of scaffolding adds nodes that are easy to get right and impossible to see.

We measure the split structurally, from the parsed files alone. A node is visible if any byte inside its extent lies outside every one of its children—that is, if it contributes text of its own to the rendering. The definition depends only on the input, never on any diff of it, which is what lets it stand in this section: two different tools disagreeing about a file still agree exactly on which of its nodes are visible. The alternative—asking which nodes a given tool's output actually displayed—is not usable as a denominator, because it moves with the tool. A diff that renders very coarsely has almost nothing visible and therefore almost nothing it can visibly get wrong. One fixture in this corpus, whose grammar fails on it and collapses 32,682 nodes into two rendered spans, would score zero visible mismatches by that definition while holding 124 real ones.

RA3 (Visibility)

31.8% of the AST nodes in this corpus carry no text of their own and cannot appear in a rendered diff at all: 68.2% are visible, pooled over 468 fixtures. The two readings agree, which is the useful part—the per-fixture median is 67.2% visible, with 60.2% and 72.8% at the tenth and ninetieth percentiles, so this is a property of source code generally rather than of a few large outliers. Roughly one AST node in three is scaffolding, and a node-level accuracy metric spends a third of its weight on decisions no reviewer can see.

The consequence is not that node-level metrics should be abandoned. It is that a rate over all nodes and a rate over visible nodes answer different questions, and a paper reporting one should say which. This paper reports both wherever it reports an accuracy figure, and Section 6 avoids the issue entirely by scoring the established tools at the line level, which is the only granularity all four of them share.

Rendering. RQ3 narrows the question to what a reader sees. RQ4 asks whether *that* has a unique answer. It is the natural place to hope the ambiguity of RQ2 washes out: several mappings may be equally correct, but perhaps they project onto the same highlighted text, so a tool that picks any of them renders the same diff and the choice never reaches the reader. The painted portion of the corpus (Section 4) is what tests that hope, because it records the rendering directly instead of deriving it from a mapping.

It does not wash out. Of the 43 painted fixtures, 19—44.2%—carry two paintings, the annotator's finding that both are correct renderings of the same change; the remaining 24 carry one, the annotator's finding that this change renders one way. Both cells are findings. A single painting is not a fixture nobody got round to painting twice, and this is worth stating because the two are indistinguishable in a file listing: the annotation tool records a single painting only where the painter examined the change and judged its rendering unambiguous.

The fork is systematic rather than a matter of care, and it has a name. Across the doubly-painted fixtures the *Minimal* paintings hold 171 entries against 209 for *Full*; of the 103 spans *Full* paints that *Minimal* does not, 40—39%—hold punctuation alone, a bracket or a comma and nothing else. The two styles are not two degrees of thoroughness. They are two consistent answers to one question: whether the brackets and separators that a change drags along with it are part of what changed. A tool that leaves a bracket unpainted disagrees with *Full* and agrees with *Minimal*, and both are on file precisely so that this is not scored as an error.

This is where the corpus can also say something the mapping format cannot. A painted entry carries a list of spans per side, so an N:M correspondence is expressible rather than approximated, and in the painted subset it is not rare: 9 of the 141 matched entries are N:M rather than one-to-one, 6.4%, in shapes from two-to-one to three-to-two. RA2 reports the same phenomenon on the mapping side as an existence result with no denominator, because nothing there can express it. Here there is a denominator, and it says that roughly one matched region in sixteen corresponds in a way no one-to-one node mapping can represent.

RA4 (Rendering)

Settling the mapping does not settle the rendering. Of the 43 changes whose rendering was painted by hand, 44.2% (19) admit two equally correct renderings, differing systematically over whether punctuation dragged along by a change counts as part of it: 39% of the spans separating the two styles hold punctuation alone. Line-based and syntax-aware tools alike are therefore scored, on those changes, against one of two right answers—and on the same subset, 6.4% of matched regions correspond N:M, which no one-to-one node mapping can express at all.

Two limits bound that rate, and they run in opposite directions. The painted subset is not a random sample: painting is manual, and it was done on the hand-written portion of the corpus first, so every painted fixture is a small, curated example of a single change pattern—median 14 lines, at most 52, across 7 of the corpus's 24 languages. That is a real restriction on what the rate describes. But it is a restriction toward the conservative side of the claim being made: a minimal example of one change pattern offers the least room for two renderings to diverge, and the rate is measured there. Whether it holds on the corpus's large, multi-site real-world changes is open, and the direction we would expect is upward. The second limit is the one RA2 already names: a painting records what an annotator noticed, so a change whose second rendering nobody saw is filed as unambiguous.

6 HOW WELL EXISTING TOOLS DO

With the ground truth characterized, and with RA2, RA3 and RA4 bounding what it can be asked, this section measures what existing tools recover from it, and what they cost to do so.

RQ5 (Tool accuracy). How accurately do current state-of-the-art code-diffing tools recover real-world code changes,

measured against human-authored ground-truth mappings of those changes?

RQ6 (Tool cost). What does each of those tools cost to run on the same corpus, and how does that cost distribute across easy and hard changes?

Comparison tools. We measure four established tools on the 468-fixture corpus: GumTree (v4.0.0-beta8) [4], Unix `diff`, and the two tree-sitter-based tools from Section 2, difftastic [5] and diffsitter [3]. CodeDiff is deliberately absent from this comparison: RQ5 asks what the state of the art recovers, and that question is answered without reference to the tool this paper introduces. CodeDiff's own accuracy is reported separately, in Section 7, against the same ground truth but not as a ranking against these four.

This comparison measures one specific property: agreement with the human-authored ground-truth mapping above. difftastic and diffsitter deliberately optimize for something this metric does not capture, a diff that reads comfortably to a human, not a mapping that reproduces a ground truth line for line, so a lower score here reflects a different design goal, not a defect. GumTree's broader language and backend flexibility likewise stays valuable well beyond what this single corpus measures. One consequence of that flexibility is visible in the numbers below: every generator GumTree offers is included here, and all but two of them are marked "Testing" rather than "Stable" by GumTree's own documentation, so its rate reflects a deliberately wide language scope rather than its performance on the backends it considers ready. Read what follows as a measurement of where each tool's design goals place it on this one metric, not as a ranking of which tool is best.

Line-level agreement. None of the four tools report an AST-node mapping, so we compare them on a common ground: which source lines each marks as touched, against the same human mapping (Table 4, Table 5). Coverage varies widely: diffsitter's compiled-in grammar set matches 306 of the 468 fixtures' languages, GumTree's generator set matches 376, difftastic's 471, and only Unix `diff` covers all 468. Each tool's own row is scored only on its own applicable subset, not padded with unscored fixtures. Because those subsets differ so much, the table also reports every tool on the 262 fixtures all four scored, which holds the fixture set fixed at the cost of over-representing the mainstream languages every tool supports.

RA5 (Tool accuracy)

Across the four tools (Table 4), line-level mismatch rates against the human mapping range from 1.111% (Unix `diff`) to 4.846% (GumTree). Every tool agrees with the human mapping on the great majority of lines, so what separates them is the small, structurally interesting remainder—and on that remainder no AST-aware tool among the four beats plain line-based `diff`, on either basis. Syntax-awareness alone does not guarantee that a tool recovers a change the way a human reads it.

Table 4: Line-level mismatch rate against human ground truth, for the four established tools, on each tool's own applicable subset of the 468-fixture corpus and on the 262 fixtures every tool scored.

Tool	Fixtures	Mismatched lines	Rate	Common
Unix `diff`	493	6,810	1.111%	0.799%
diffsitter	306	6,247	1.377%	1.046%
difftastic	471	10,101	1.769%	1.896%
GumTree	376	18,344	4.846%	5.512%

Table 5 reports the same agreement per fixture rather than pooled over lines, and the two readings weight the corpus differently: a pooled rate is dominated by a handful of very large fixtures, while the buckets weight every change equally. On the pooled rate the four tools span 1.13% to 4.85%; by fixtures mapped with no mismatched line at all they span 50% down to 34%. The table also groups the tools by whether their output carries sub-line detail at all, which is a statement about what each tool reports, not about how it was scored here – every number in this section is line-level.

Two details behind that answer are worth stating. The first is how wide the agreement is: even the highest mismatch rate leaves more than 95% of lines in agreement, so the question that separates tools is a narrow one. The second is the qualification RA2 and RA4 already attach to every number in the table: on the changes where the ground truth admits more than one correct mapping, or more than one correct rendering, a tool that picked a different valid answer can be recorded here as wrong. For the mapping ambiguity the corpus removes that error, by accepting any consistent pairing within a group. For the other two it does not. These rates score which *lines* each tool marks as touched against the human mapping—the only granularity all four tools share—so the paintings, which are the only place the corpus records a second correct rendering, never enter the comparison at all; RA4's error term and RA2's N:M term both sit in the tables below unremoved and unmeasured. Neither can be signed: they may run in a tool's favour or against it.

Cost. RQ6 asks the same four tools a different question. Table 6 reports wall-clock percentiles over the same fixtures, sorted fastest to slowest by median. We report GumTree twice: once invoked as a fresh process per fixture, the realistic cost of a single CLI invocation, and once run inside one persistent JVM across all fixtures, isolating JVM startup from GumTree's own matching cost.

RA6 (Tool cost)

The four tools span three orders of magnitude at the median, from Unix `diff`'s 2.6 ms to GumTree's 519.2 ms per invocation, and the spread widens rather than narrows into the tail: at the 99th percentile the same range runs from 11.8 ms to 12485.5 ms. Every AST-aware tool has a tail heavier than its median suggests—the ratio between

Table 5: Per-fixture agreement with the human mapping, bucketed. Every number is *line-level* agreement, including for the tools grouped as reporting sub-line detail – that grouping describes what each tool's output carries, not how it was scored. "Perfect" means zero mismatched lines, not a rounded 100%. Each tool is scored on its own applicable subset (n), so percentages, not counts, are comparable across rows.

Tool	n	Perfect	≥99%	95–99%	<95%
Line granularity only					
UNIX diff (baseline)	493	244 (49%)	138 (28%)	64 (13%)	47 (10%)
git (Myers)	493	246 (50%)	141 (29%)	59 (12%)	47 (10%)
git (minimal)	493	246 (50%)	141 (29%)	59 (12%)	47 (10%)
git (patience)	493	245 (50%)	140 (28%)	61 (12%)	47 (10%)
git (histogram)	493	245 (50%)	140 (28%)	61 (12%)	47 (10%)
Reports sub-line detail (not exercised by this line-level metric)					
BDiff (per process)	493	240 (49%)	134 (27%)	70 (14%)	49 (10%)
nvim -d	493	245 (50%)	141 (29%)	60 (12%)	47 (10%)
diffsitter	306	103 (34%)	95 (31%)	64 (21%)	44 (14%)
difftastic	471	255 (54%)	92 (20%)	61 (13%)	63 (13%)
GumTree (binary)	376	181 (48%)	75 (20%)	63 (17%)	57 (15%)

Table 6: Wall-clock time percentiles for the four established tools, milliseconds, sorted fastest to slowest by median. CodeDiff is reported separately in Section 7, for the same reason it is absent from Table 4.

Tool	p50	p90	p99
Unix `diff`	2.6	4.5	11.8
diffsitter	8.6	63.3	239.1
difftastic	53.3	171.6	1842.9
GumTree (warm JVM)	66.2	722.6	5293.4
GumTree (cold, per-invocation)	519.2	1622.1	12485.5

a tool's own p99 and p50 is at least 20× for each of the three, against under 5× for Unix `diff`—so cost on this corpus is governed by the hard minority of changes, not by the common case.

The two GumTree rows bracket how much of its cost is JVM startup rather than matching. A warm JVM takes its median from 519.2 ms to 66.2 ms, so most of the per-invocation figure is process startup, not GumTree's algorithm. We report both because each answers a different question: the cold number is what a developer running a diff from the command line actually waits for, and the warm number is what GumTree's matching costs once that is amortized away.

These percentiles must be read against the right population. The fixture corpus is not the distribution of real commits, in which 47.5% touch 10 lines or fewer [1]: it deliberately concentrates hard, structurally interesting changes, because easy changes exercise nothing worth measuring. Every tail figure here is therefore a stress reading on an adversarially weighted sample rather than an estimate of what a developer waits for on a typical commit, and the medians are the closer of the two to that.

Table 7: Timing noise per series: per-fixture coefficient of variation across repeated measurements of the same fixture (lower means a single run is more trustworthy on its own).

Series	n	Median CoV	p90 CoV
TreeSitter parse (lower bound)	486	3.3%	18.1%
UNIX diff (baseline)	486	7.4%	21.4%
git (Myers)	486	2.6%	19.0%
git (minimal)	486	1.9%	17.1%
git (patience)	486	2.0%	16.2%
git (histogram)	486	1.9%	16.9%
BDiff (per process)	486	2.2%	8.4%
nvim -d	486	11.5%	23.8%
BDiff (warm interpreter)	486	1.6%	5.3%
CodeDiff	486	2.6%	12.7%
GumTree (binary)	373	2.1%	11.3%
GumTree (warm JVM)	373	0.5%	5.1%
diffsitter	303	1.5%	5.9%
difftastic	465	0.8%	9.0%

Table 7 reports a companion measurement: how much any single timing run can be trusted, the per-fixture coefficient of variation across repeated measurements of the same fixture. This is what motivated repeating every timing number in this paper across multiple runs rather than trusting a single measurement.

7 CODEDIFF

Everything above is about the problem. This section is the paper's system contribution: a syntax-aware diffing tool built against those measurements. It is presented last, and evaluated against the same ground truth as Section 6's four tools but not as a ranking against them, for the reason Section 6

gives—each of those tools optimizes for something this corpus does not measure.

The measurements in Section 3 give CodeDiff two concrete, named design targets, stated in place of an asymptotic bound. **Robust**: CodeDiff must handle every file up to the measured maximum, 573,334 AST nodes, with headroom above that number. **Fast**: CodeDiff must compute a diff in under 400 ms for 99.99% of real-world commits, the range that covers the overwhelming majority of commits per Arafat and Riehle's measurement. The rest of this section reports how well CodeDiff meets both targets in practice, and how accurately it maps the corpus of Section 4.

CodeDiff matches two ASTs in five phases. Each phase either resolves part of the mapping directly or narrows the residual left for the next phase. Later phases only ever see nodes earlier phases left unmatched. A sixth step closes the mapping, recording the deletion or insertion implied by whatever every phase above left undecided.

Phase 1, hash-based descent. CodeDiff matches subtrees by two hash variants, in order: byte-identical subtrees, then same-shape subtrees with any leaf value. Each variant claims what it can before the next runs. This phase is CodeDiff's own design, motivated directly by Section 3's commit-size data: since most commits touch a small fraction of a file's lines, most of a typical file's AST is byte-identical between before and after. Resolving that identical majority with cheap hashing, before any tree-edit-distance computation runs at all, avoids paying that computation's cost on content that never changed.

Phase 2, contextual exact matching. Comments and modifiers (attributes, decorators) that immediately precede an already-matched node, and diagnostic statements (logging calls, assertions, panics) that are byte-identical on both sides, get matched directly. Both are cheap, high-confidence matches that reduce the work left for the tree-edit-distance phases below. Neither is reachable by phase 1's structural hashing: comments and modifiers are not part of the hashed AST shape, and diagnostic statements are held back deliberately, so that matching one does not fragment a larger match around it.

Phase 3, syntax-aware subtree matching. This phase generalizes three matching strategies into one engine: matching by fully-resolved name (handling overloads and duplicate names across scopes), a greedy cost-estimate matcher for containers with no name structure, such as an `if` block or a loop body, and an overlap matcher pairing import statements that share a base path. Before any of these run, large flat subtrees, such as a macro's token list or a big flat argument list, are pre-matched directly with Myers' algorithm [7], since a flat sequence has no tree structure worth exploiting—exactly the structures that drive Section 3's measured AST-size tail. Each accepted pair is then handed to APTED [8], scoped to that pair alone, to compute its exact internal edit script. This is the only place a tree-edit-distance computation runs at all, and the scoping is what keeps it affordable: APTED sees one accepted container pair at a time, never the whole file.

Phase 4, residual alignment. Whatever remains unmatched after phases 1 through 3 is resolved by a Myers-LCS alignment [7] over the residual forest, bracketed by two passes of strict bottom-up propagation: a node whose children all match the children of exactly one node on the other side is matched to it, which shrinks the residual the alignment must guess at, and is run again afterward so that pockets the alignment just resolved can still bubble up to their now-complete ancestors.

Earlier versions of CodeDiff instead ran one whole-residual APTED computation here, with the Myers-LCS alignment substituting only when that computation looked too expensive. That design was abandoned: its cost is driven by residual *shape* rather than size, which makes it impossible to gate reliably. Two measured cases make the point—an 11,647-node Vimscript residual took 87 seconds, while a 258,504-node JSON residual took 9.4. No size threshold separates those two, so the exact computation was removed from the whole-file path entirely and survives only in phase 3's scoped form. This is a direct consequence of the size tail in Section 3: an unbounded whole-residual computation cannot meet the 400 ms target at any threshold.

Phase 5, move-detection recovery. The final matching pass. Ordered tree edit distance cannot express a subtree that relocated across a matched boundary as anything but a delete plus an insert. This phase, adapted directly from GumTree's recovery-mappings phase [4], pairs up whole subtrees left fully deleted on one side and fully inserted on the other, when they are byte-identical and large enough to rule out coincidence. It runs last by necessity rather than by preference: every phase above requires some anchor—a hash, a name, a shared matched ancestor—before it will consider a pair at all, and content that moved between two containers that both changed identity has none. It is also, of the pipeline's optional passes, the one whose absence costs the most accuracy: a relocated subtree is the single shape ordered tree edit distance cannot express except as a delete plus an insert, so nothing downstream recovers it.

Zhang-Shasha [9] plays no role in this pipeline as a shipped algorithm. CodeDiff's own test suite instead implements it from scratch as an independent oracle, and fuzz-tests thousands of random trees to confirm that APTED's result always matches Zhang-Shasha's, bit for bit. This checks the pipeline's most correctness-critical component against a second, independently-implemented ground truth, not only against itself.

Accuracy. Measured directly against the human mapping, CodeDiff maps 4,956,544 of 4,959,531 AST nodes correctly across the 468 fixtures, 2,987 mismatches, or 99.94% node-level accuracy. That denominator counts every AST node, including every ancestor of every change up to the root, so it partly reflects how deeply a given grammar nests. Restricted to the 68.2% of nodes RA3 finds visible—those carrying text of their own, and therefore able to reach the screen when the diff is rendered—the figure is 99.94%, 2,037 mismatches out of 3,380,690. The two rates agreeing to two decimal places is itself the useful result, and it is exactly the check RA3 argues

every node-level accuracy claim should carry: CodeDiff's residual errors are not concentrated in the invisible third of the tree, so the headline figure is not being flattered by nodes no reviewer ever sees.

Speed. Against the four tools of Table 6, CodeDiff is the fastest AST-aware tool at the median, 7.0 ms against diffsitter's 8.6 ms and difftastic's 53.3 ms, but that ordering does not hold across the distribution. diffsitter, whose narrower hand-picked grammar set does markedly less work, overtakes it at the 90th percentile (63.3 ms against 137.7 ms) and is an order of magnitude ahead at the 99th (239.1 ms against 2177.4 ms); difftastic's tail is heavier still in aggregate cost and essentially ties CodeDiff at the 99th percentile. Unix `diff` remains fastest at every percentile, since it never parses or compares tree structure. CodeDiff's tail is heavy, and the design above explains why: the phases that resolve the common case cheaply do nothing for a large, genuinely dissimilar residual, which is exactly what the slowest fixtures in this corpus are.

Read against the Fast target, the percentiles need the population qualification RA6 already attaches to the same measurements. The target—400 ms for 99.99% of real-world commits—is a claim about the real distribution of commits, in which 47.5% touch 10 lines or fewer [1], and the fixture corpus is deliberately not that distribution. CodeDiff's 99th percentile on this corpus (2177.4 ms) is therefore a stress figure on an adversarially weighted sample, not the target metric; its median (7.0 ms) and 90th percentile (137.7 ms) on the same hard sample sit well inside the budget. Validating the 99.99% claim on a uniformly drawn commit sample remains future work, and we state it here as a design target rather than a measured result.

Robustness. We sampled 925 real (repository, commit, file) pairs from real Rust repositories and ran every pair through CodeDiff, up to a 16,000 combined AST-node cap and a 120-second timeout per pair. Of the 925 sampled pairs, 377 were unavailable locally because the repository's history had since been rewritten, not a CodeDiff failure, and 142 exceeded the node cap and were skipped before CodeDiff ever ran. CodeDiff completed all 406 remaining pairs with zero panics and zero timeouts. This is a sampled, bounded-scale result, not a claim over the full 100-repository corpus, and the 142 pairs above the node cap are precisely the inputs the Robust target is about, so they are excluded from the evidence rather than counted in its favor. Within those limits the result is consistent with CodeDiff's stated Robust target in this section.

Summary. Combining established techniques rather than inventing new ones, CodeDiff reaches 99.94% AST-node accuracy against human-verified ground truth—99.94% on the nodes a reader can see—a median well inside its interactive budget, and zero panics or timeouts across the sampled robustness evaluation. Building a production-grade syntax-aware diffing tool did not require a new matching algorithm, only careful combination and measurement of existing ones, against a problem measured first.

8 RELATED WORK

Tree-diffing tools. GumTree [4] remains the reference point for source-code tree diffing, and CodeDiff's move-recovery phase adapts its recovery-mappings heuristic directly (Section 7). The difference is one of goal: GumTree targets research use, with broad language and backend flexibility, while CodeDiff narrows to a tree-sitter-only frontend and spends the saved generality on production targets. difftastic [5] and diffsitter [3] share CodeDiff's parsing layer but optimize for a diff a human reads comfortably rather than for fidelity of the underlying node mapping; RA5 reflects that difference in design goal, not a defect in either tool. GumTree remains the foundation CodeDiff's move-recovery phase builds on directly, and its language and backend flexibility stays valuable well beyond what RA5's single corpus measures. Unix `diff`, built on Myers' algorithm [7], remains the baseline every one of these tools must justify itself against—on our corpus it is both the fastest tool measured and, at the line level, more accurate than every pre-existing AST-aware tool compared.

Ground truth and its ambiguity. GumTree's own evaluation, and most that followed it, score a tool against a single reference mapping per change. RA2 and RA4 are an argument that this default understates what a corpus can be asked: on the changes where several mappings, or several renderings, are equally correct, a fixed reference records a correct answer as wrong. The corpus here departs from the default in two ways—a multi-map group, which admits any consistent pairing of an interchangeable set, and a second, independently authored painting where the rendering is not unique—and both were added because annotating real changes forced them, not by design. We are not aware of a code-diffing ground truth that records either.

Empirical studies. Dyer et al. [2] and Lämmel et al. [6] mine AST nodes at scale to study API usage; Arafat and Riehle [1] measure the commit-size distribution of open-source software, the distribution CodeDiff's Fast target is stated against. This paper's empirical study differs from these in purpose rather than method: it measures file- and AST-size distributions specifically to set, and then evaluate, the engineering targets of a diffing tool.

9 CONCLUSION AND FUTURE WORK

This paper's most transferable result is not about any tool. It is that the ground truth syntax-aware diffing is measured against is weaker than the metrics built on it assume, in three independent ways, each measured here: a correct mapping is often not unique (RA2), a third of the nodes a node-level metric scores are invisible to the reader (RA3), and even a settled mapping does not settle the rendering (RA4). None of these is a defect in a particular corpus. They are properties of code change, and any evaluation that scores a diff against one fixed answer carries all three as an error term whether or not it reports them. Reporting a fixed-answer metric is still defensible; reporting it alone is not.

Against that background, the tool contribution is deliberately modest. None of CodeDiff's matching algorithms

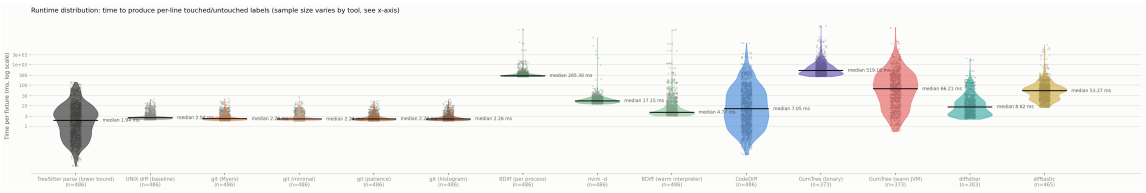

Figure 3: Full per-fixture runtime distribution (log scale), every individual repeated measurement plotted, alongside a tree-sitter-parse-only reference violin (the lower bound every AST-aware tool here must pay before its own work even starts).

are new: it combines APTED's exact tree edit distance, GumTree's move-recovery heuristic, and Myers' sequence diff into a single pipeline, engineered and measured against the real distribution of source code rather than an assumed worst case. On the corpus in this paper that combination reaches an accuracy, speed, and reliability comparable to or exceeding each individual predecessor it draws from, without displacing what any of them are independently good at: GumTree's language and backend flexibility, difftastic's and diffsitter's speed and readability, Unix `diff`'s universality.

The same discipline applies to the exact algorithm at the pipeline's core. CodeDiff began by running one whole-residual APTED computation per file, exactly as its asymptotic analysis invites, and removed it after measurement showed its cost tracks residual shape rather than residual size closely enough that no size threshold can gate it safely. What survives is APTED scoped to individual container pairs, where the same exactness is affordable. An exact algorithm is not automatically the right one to run at whole-file scale, and only measurement distinguishes the two regimes.

Based on the lessons reported in this paper, we recommend the following to builders of syntax-aware developer tooling:

- Report an accuracy rate alongside what its ground truth cannot say: how often that ground truth admits more than one correct answer, and how much of what it scores a reader can see. Both are measurable on any annotated corpus, and both bound the rate being reported.
- Let the annotation format record ambiguity rather than resolve it. Where several answers are equally correct, forcing one makes an arbitrary choice into ground truth, and every tool that chooses differently is then scored wrong for a choice that was never wrong.
- Engineer against the measured distribution of real inputs, not against the worst-case bound alone: the common case and the tail require different machinery, and both occur in practice.
- Scope an exact algorithm to the inputs it can serve within budget rather than abandoning or over-applying it: the same computation that is unaffordable over a whole file is the right tool for a bounded subtree pair.
- Keep an exact algorithm as an independent test oracle even when the shipped path is heuristic: CodeDiff fuzz-checks its APTED implementation against an independently implemented Zhang-Shasha on thousands of random trees.

Future work includes widening the robustness corpus beyond Rust and extending the accuracy comparison to more languages GumTree also supports, but the clearest directions are the ones RA2 and RA4 open. Both rates are floors: the corpus establishes that ambiguity is common, and establishes it only where an annotator noticed and recorded it. Detecting candidate ambiguity automatically, and re-annotating the 217 fixtures that predate the multi-mapping facility, would turn a floor into a census. RA4's floor is narrower still and easier to raise: painting is manual, and the 43 changes painted so far are the corpus's small curated ones, so whether 44.2% holds on large, multi-site real-world changes is simply unmeasured. The $N{:}M$ result needs more than annotation effort. The painted format can express such a correspondence and finds 6.4% of matched regions carrying one; the mapping format cannot express it at all, and until a node-level ground truth can, its prevalence there stays something found by reading rather than counted.

REFERENCES

[1] Osama Arafat and Dirk Riehle. 2009. The Commit Size Distribution of Open Source Software. In *2009 42nd Hawaii International Conference on System Sciences*. 1–8. https://doi.org/10.1109/HICSS.2009.421

[2] Robert Dyer, Hridesh Rajan, Hoan Anh Nguyen, and Tien N. Nguyen. 2014. Mining billions of AST nodes to study actual and potential usage of Java language features. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 779–790. https://doi.org/10.1145/2568225.2568295

[3] Afnan Enayet. 2020. diffsitter: A Tree-Sitter-Based AST Difftool for Meaningful Diffs. https://github.com/afnanenayet/diffsitter. Accessed 2026-07-31.

[4] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*. Västerås, Sweden, 313–324. https://doi.org/10.1145/2642937.2642982

[5] Wilfred Hughes. 2018. Difftastic: A Structural Diff That Understands Syntax. https://github.com/Wilfred/difftastic. Accessed 2026-07-31.

[6] Ralf Lämmel, Ekaterina Pek, and Jürgen Starek. 2011. Large-scale, AST-based API-usage analysis of open-source Java projects. In *Proceedings of the 2011 ACM Symposium on Applied Computing*. 1317–1324.

[7] Eugene W. Myers. 1986. An O(ND) Difference Algorithm and Its Variations. *Algorithmica* 1, 2 (1986), 251–266. https://doi.org/10.1007/BF01840446

[8] Mateusz Pawlik and Nikolaus Augsten. 2015. Efficient Computation of the Tree Edit Distance. *ACM Transactions on Database Systems* 40, 1, Article 3 (2015), 3:1–3:40 pages. https://doi.org/10.1145/2699485

[9] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. https://doi.org/10.1137/0218082